Revisão técnica do aplicativo Conflora

Problemas encontrados e melhorias sugeridas
18 de outubro de 2026 · Edmar Júnio

22 recomendações	9 urgentes · P1	68/70 no navegador

Parecer executivo

O aplicativo já atende catálogo, caixa, comandas, estoque, equipe, entregas, relatórios e atendimento com IA. A prioridade recomendada é corrigir segurança, confiabilidade das mensagens e critérios financeiros antes de ampliar funcionalidades.

Esta revisão reúne 22 recomendações: 9 de prioridade P1, 11 de prioridade P2 e 2 de prioridade P3. P1 significa correção urgente; P2, próxima etapa de estabilização; P3, evolução planejada. A prioridade considera o impacto e o cenário necessário, sem presumir que uma falha já tenha sido explorada em produção.

Os primeiros trabalhos devem ser R01 (HTML inseguro), R02 (senhas e PINs), R04 (retorno a usuários locais após falha no banco), R05 (PDF financeiro público) e R06 (data dos recebimentos). R03 e R07–R09 completam a primeira etapa de proteção e confiabilidade.

Foram reproduzidos localmente: execução de código no navegador do administrador a partir do nome de um cliente; armazenamento de senha em texto; PDF financeiro sem autenticação; retorno de administrador padrão após leitura negada; recebimento contabilizado no mês de criação da venda; descarte de uma mensagem ao repetir um processamento que falhou; tarefa aceita sem segredo configurado; e webhook aceito sem processamento quando a fila está indisponível.

Resultado dos testes: 234 de 236 testes gerais aprovados; 68 de 70 testes de navegador aprovados. A verificação de sintaxe e o lint passaram na conferência desta sessão. A auditoria das dependências de produção apontou 8 entradas moderadas, relacionadas à mesma cadeia de dependências, sem entradas altas ou críticas no resultado consultado.

Nenhuma melhoria foi aplicada ao aplicativo. Os testes e as reproduções usaram dados isolados; não foram realizados pagamentos nem enviados avisos reais. O relatório recomenda ações e critérios para a execução posterior.

Arquitetura e abrangência

Fluxo principal: catálogo ou caixa → rotas Express → serviços de pedidos → transações do Firestore → integração Pix → confirmação e estoque → fila de notificações → WhatsApp e relatórios. A vitrine de múltiplas lojas possui rotas, autenticação e contratos próprios em src/storefront.

A revisão percorreu as áreas de autenticação e sessão, autorização por cargo, catálogo e imagens, checkout, Pix e fiado, estoque, entregas, notificações, relatórios, IA, cache, interface, publicação e testes. O inventário consultado reuniu 203 arquivos em src, public, scripts e test, excluindo bibliotecas de terceiros e código gerado. A análise aprofundou os pontos de entrada e os fluxos críticos; não representa leitura integral de cada linha desses 203 arquivos.

Há decisões corretas a preservar: transações e reservas de estoque no PDV; conferência bancária de txid, chave, valor e devoluções; idempotência nas operações do caixa; comprovantes privados vinculados à operação e ao usuário; separação dos cargos; cookies HttpOnly; assinatura de webhook; isolamento dos testes; e testes de largura, entregas, pagamentos divididos e navegabilidade.

A execução deve manter essa direção: validação na entrada, regras de negócio em serviços, persistência transacional e efeitos externos após o registro durável. Evite reescrever o app inteiro ou trocar o banco apenas para resolver os problemas listados.

As referências abaixo são relativas à raiz C:/Users/Junin/confloraai, na revisão Git 209edfec5f98da75abfb68fe03cee22b540e55ed. Não houve inspeção das permissões reais do Google Cloud, regras publicadas, backups ativos ou comportamento do domínio em produção. Essas condições exigem validação própria antes da publicação de correções.

R01 — Remover a execução de HTML vindo de cadastros

Prioridade: P1 · Evidência: reproduzido · Área: segurança e administração.

Problema e impacto: a lista de equipe insere nome e e-mail diretamente em innerHTML. O cadastro público aceita um nome contendo HTML. Ao abrir a equipe, um manipulador de evento desse nome executou no navegador autenticado do administrador. Mesmo com cookie HttpOnly, esse código pode realizar requisições com a sessão aberta.

Melhoria sugerida: construir células com textContent e elementos DOM; remover interpolação de dados em atributos onclick; aplicar a mesma revisão às listas de operadores e consulta de preços. Adicionar política de segurança de conteúdo como proteção complementar, após tratar os pontos de inserção.

Critério de aceite: nomes com marcação aparecem como texto nas telas; nenhum evento é executado; a regressão deve passar também com nome alterado pelo próprio cliente.

Referências: src/http/views-admin.js:1273, :1364 e :1252; src/database/firestore.repository.js:2640; public/storefront/team-drawer.js:147.

R02 — Proteger senhas e PINs armazenados

Prioridade: P1 · Evidência: reproduzido e leitura do código · Área: autenticação.

Problema e impacto: registerUser salva password diretamente e loginUser/authenticateUser fazem comparação textual. O login por e-mail também aceita o PIN do colaborador. A senha mínima do cadastro principal é de quatro caracteres. Uma exposição do cadastro revela as credenciais, e o PIN curto pode servir de senha remota.

Melhoria sugerida: migrar para hashes com salt e derivação resistente a tentativas, aproveitando o padrão scrypt já existente em src/storefront/auth.js; separar PIN operacional de login remoto; planejar migração das contas existentes e recuperação de senha. Não basta esconder o campo nas respostas da API.

Critério de aceite: nenhuma nova senha ou PIN fica recuperável no banco; registros antigos são migrados ou exigem redefinição; a sessão e os cargos continuam funcionando após a migração.

Referências: src/database/firestore.repository.js:2606, :2643, :2698 e :2809; src/storefront/auth.js:15.

R03 — Limitar tentativas e consumo nas rotas públicas

Prioridade: P1 · Evidência: leitura dos fluxos · Área: autenticação e custo de IA.

Problema e impacto: as rotas principais de login, cadastro e PIN não têm limitador de tentativas no aplicativo. A listagem pública fornece identificadores de operadores. A consultoria botânica também pode acionar IA sem uma cota própria. A vitrine por tenant já tem limitador, mas ele não protege essas rotas principais.

Melhoria sugerida: limitar por conta e origem, com armazenamento compartilhado entre instâncias, espera progressiva e eventos de auditoria; limitar tamanho e frequência das consultas de IA. Evitar bloqueio permanente de uma conta apenas por tentativas de terceiros. Uniformizar erros de credenciais para reduzir enumeração.

Critério de aceite: tentativas excessivas recebem 429 com orientação de espera; a restrição persiste entre instâncias e não permite contornar o limite alternando login por e-mail e PIN.

Referências: src/http/app.js:1484, :2519, :2535, :2568 e :2584; src/storefront/routes.js:145.

R04 — Impedir retorno a usuários padrão após falha no banco

Prioridade: P1 · Evidência: reproduzido · Área: persistência e autenticação.

Problema e impacto: getAllUsers, ao receber PERMISSION_DENIED, desabilita o cliente Firestore e retorna a coleção local. O construtor preenche essa coleção com operadores e PINs fixos, inclusive administrador, mesmo fora dos testes. A reprodução confirmou o retorno desse administrador. Isso altera a fonte de autorização em uma falha de infraestrutura; não foi demonstrado um login completo em produção por esse caminho.

Melhoria sugerida: criar usuários de demonstração somente em ambiente explicitamente isolado; em produção, bloquear autenticação e operações dependentes com erro de indisponibilidade; eliminar o estado global que troca silenciosamente o banco por memória. Revisar readStoreSettings pelo mesmo efeito global.

Critério de aceite: erro de permissão nunca retorna operadores de exemplo nem permite gravação apenas local; a aplicação informa indisponibilidade e registra o incidente.

Referências: src/database/firestore.repository.js:102 e :2480; src/storefront/existing-store.js, função readStoreSettings.

R05 — Tornar privados os PDFs de fechamento

Prioridade: P1 · Evidência: reproduzido · Área: dados financeiros.

Problema e impacto: /reports publica a pasta de relatórios sem autenticação, e o nome do arquivo contém apenas a data. Uma requisição sem sessão ao fechamento de 18/10/2026 retornou 200 e application/pdf. A pasta também está dentro do diretório public do Firebase Hosting, portanto proteger apenas a rota Express é insuficiente.

Melhoria sugerida: retirar os arquivos financeiros de public e do histórico distribuído; armazenar em local privado e fornecer download autenticado com escopo de cargo e operador. Para compartilhamento autorizado, usar autorização temporária e expiração. Definir retenção e revogação.

Critério de aceite: visitante não baixa o relatório pelo Cloud Run nem pelo Hosting; vendedor/caixa acessa somente o escopo permitido; o arquivo não é incluído nos ativos públicos da publicação.

Referências: src/http/app.js:1193; src/reports/daily-report-pdf.js:27; firebase.json, hosting.public; public/reports/fechamento-caixa-2026-10-18.pdf.

R06 — Separar data da venda e data do recebimento

Prioridade: P1 · Evidência: reproduzido · Área: painel gerencial.

Problema e impacto: o resumo de período seleciona pedidos por createdAt e soma o valor como recebido conforme o estado atual do pedido. Em uma venda criada em 31/01 e paga em 01/02, o cálculo atribuiu R$ 100 de entrada a janeiro e R$ 0 a fevereiro. A consulta do repositório também busca pedidos pela criação, excluindo recebimentos de vendas antigas do período atual.

Melhoria sugerida: definir explicitamente o fato de venda e o fato de recebimento. Consultar movimentos financeiros pela data efetiva do recebimento, inclusive parcelas, Pix e quitação de fiado. Manter critérios e fuso iguais no painel e nos PDFs.

Critério de aceite: o exemplo mantém a venda no período definido pela regra comercial e registra a entrada somente em fevereiro; pagamentos parciais de meses diferentes reconciliam com os movimentos.

Referências: src/reports/period-executive-metrics.js:32, :54 e :131; src/database/firestore.repository.js:1176 e :1208.

R07 — Persistir mensagens antes de confirmar o webhook

Prioridade: P1 · Evidência: reproduzido e leitura do fluxo · Área: WhatsApp e fila.

Problema e impacto: o webhook responde 200 antes de persistir a mensagem ou garantir o envio à fila. Se o cliente Cloud Tasks existir, mas não estiver configurado, enqueueProcessMessage retorna { inProcess: true }; o chamador ignora esse retorno e não processa a mensagem. O teste recebeu 200 e contabilizou zero mensagens processadas. Queda do processo após a resposta também pode perder atendimento.

Melhoria sugerida: registrar um evento de entrada durável ou confirmar a criação da tarefa antes do ACK; tratar explicitamente fila indisponível; proibir fallback volátil em produção. Preparar reprocessamento e fila de falhas.

Critério de aceite: falha de configuração, falha ao enfileirar e reinício após o recebimento preservam a mensagem para repetição; nenhuma resposta de sucesso confirma uma mensagem descartada.

Referências: src/http/app.js:1263 e :1305; src/integrations/task-queue.client.js:43.

R08 — Corrigir a idempotência do atendimento

Prioridade: P1 · Evidência: reproduzido · Área: processamento de mensagens.

Problema e impacto: a mensagem é marcada como processada antes do histórico, da resposta e do envio. Na reprodução, uma falha ao salvar o histórico interrompeu o processamento; a repetição retornou DUPLICATE e não retomou o atendimento. A consulta e a marcação também são operações separadas, permitindo concorrência entre trabalhadores.

Melhoria sugerida: reservar atomicamente o evento com estado PROCESSING e prazo de posse; marcar COMPLETED somente após registrar o resultado; permitir retomada de falhas. Registrar a saída em outbox para separar o envio de WhatsApp do processamento de negócio.

Critério de aceite: duas instâncias não executam o mesmo efeito simultaneamente; uma falha intermediária permite repetição; reenvios não criam pedidos nem respostas comerciais duplicadas.

Referências: src/services/message.service.js:61 e :66; src/database/firestore.repository.js:2026 e :2051.

R09 — Bloquear tarefas quando o segredo não está configurado

Prioridade: P1 · Evidência: reproduzido sob condição de configuração · Área: endpoints internos.

Problema e impacto: /tasks/processar-mensagem só verifica o segredo se TASK_SECRET estiver preenchido. Com o assistente habilitado e segredo vazio, uma chamada sem autenticação foi aceita com HTTP 200. Dependendo das integrações ativas, isso pode consumir IA e provocar atendimento para números enviados pelo chamador.

Melhoria sugerida: impedir inicialização do recurso sem autenticação configurada; exigir autenticação em toda chamada e preferir identidade verificável da tarefa. Aplicar o mesmo inventário de proteção às rotas internas e aos webhooks.

Critério de aceite: tarefa sem credencial é recusada mesmo quando a configuração está incompleta; requisições legítimas da fila continuam aceitas; ambiente de teste usa credenciais próprias.

Referências: src/http/app.js:1413 e :1418; src/config/env.js:69; src/integrations/task-queue.client.js:53.

R10 — Garantir unicidade de e-mail com transação

Prioridade: P2 · Evidência: leitura de sequência não atômica · Área: cadastro e vínculo financeiro.

Problema e impacto: o cadastro consulta todos os usuários para procurar e-mail e grava depois. O caixa verifica e-mail de cliente antes da transação que salva customers. Duas solicitações simultâneas podem passar pela verificação. O vínculo financeiro do cliente exige exatamente um perfil por e-mail, portanto a duplicidade pode impedir a consulta ou quitação do saldo. IDs baseados só em Date.now também podem colidir.

Melhoria sugerida: criar um índice de identidade por e-mail normalizado, reservar esse vínculo na mesma transação e usar identificadores aleatórios. Definir um procedimento de consolidação para registros legados duplicados.

Critério de aceite: cadastros concorrentes do mesmo e-mail produzem um único vínculo; a segunda operação retorna conflito; Google, senha e caixa resolvem a mesma identidade sem alterar o cargo.

Referências: src/database/firestore.repository.js:2621 e :2639; src/http/pdv.routes.js:150 e :154; src/orders/customer-debt-pix.service.js:7.

R11 — Só confirmar fechamento e exclusão após persistência

Prioridade: P2 · Evidência: leitura do tratamento de falhas · Área: administração e caixa.

Problema e impacto: closeCashierDay captura falha ao salvar o fechamento, segue com memória local e retorna success: true. deleteUser também ignora erro do Firestore e pode informar exclusão concluída. O operador recebe confirmação de uma alteração que pode desaparecer no reinício ou continuar existindo no banco.

Melhoria sugerida: falhar explicitamente quando a gravação obrigatória não ocorrer; distinguir fechamento salvo, PDF gerado e envio pendente. Usar chave idempotente para o fechamento e guardar o documento final em armazenamento durável. Preferir desativação auditada de colaboradores.

Critério de aceite: indisponibilidade do banco não produz sucesso; repetir a operação não duplica fechamento; a consulta posterior confirma o estado persistido, inclusive após reinício do serviço.

Referências: src/database/firestore.repository.js:1221, :1250 e :2762; src/reports/daily-report-pdf.js:27.

R12 — Restringir o acompanhamento em tempo real às comandas necessárias

Prioridade: P2 · Evidência: leitura de consulta e escopo · Área: desempenho e eventos.

Problema e impacto: watchOpenOrders inclui estados CONFIRMED, PAID_CONFIRMED e DELIVERED sem filtro de data ou paginação. O listener é criado por conexão de navegador. Com o crescimento das vendas, cada abertura pode ler grande parte do histórico. O filtro de emissão restringe entregadores, mas não filtra IDs por autoria para vendedores.

Melhoria sugerida: separar fila ativa e histórico, limitar janela temporal e paginar o histórico; aplicar o mesmo escopo de autorização do detalhe da comanda ao stream. Compartilhar listeners por instância quando adequado e medir volume de leituras.

Critério de aceite: uma loja com histórico grande carrega apenas a fila relevante; vendedor não recebe IDs de vendas fora do seu escopo; desconectar o navegador encerra a assinatura.

Referências: src/orders/pdv.repository.js:75; src/http/pdv.routes.js:168 e :179.

R13 — Buscar o usuário da sessão diretamente pelo ID

Prioridade: P2 · Evidência: leitura do middleware · Área: custo e latência.

Problema e impacto: cada requisição autenticada busca a sessão e depois executa getAllUsers para localizar uma pessoa. Em produção, isso lê a coleção inteira de usuários. Chamadas de atualização, produtos, favoritos e relatórios multiplicam o custo conforme cresce a base de clientes.

Melhoria sugerida: adicionar consulta direta ao documento de usuário; manter a verificação de ativo e cargo atuais. Para login por e-mail, usar índice de identidade. Se houver cache, definir invalidação para desativação e alteração de permissão.

Critério de aceite: resolver uma sessão tem quantidade de leituras independente do número total de usuários; desativar um colaborador corta o acesso no prazo definido e testado.

Referências: src/security/web-session.js:77; src/database/firestore.repository.js:2480; src/http/app.js:1108.

R14 — Unificar os nomes do serviço na publicação

Prioridade: P2 · Evidência: inconsistência nos arquivos · Área: entrega e operação.

Problema e impacto: scripts/deploy.sh publica por padrão o serviço conflora-api, enquanto firebase.json encaminha requisições para confloraai. O projeto padrão do script também difere do padrão principal do backend. Usar esse roteiro sem ajustes pode publicar uma versão em um serviço que o site não acessa.

Melhoria sugerida: definir uma única configuração por ambiente para projeto, serviço, região, URLs e fila; validar o destino antes de publicar; incluir teste de fumaça pelo domínio do Hosting e identificação da revisão servida. Consolidar os roteiros legados e os de desenvolvimento.

Critério de aceite: o procedimento documentado publica a revisão esperada no serviço referenciado pelo Hosting; a rota de saúde informa a mesma revisão; o rollback tem destino explícito.

Referências: scripts/deploy.sh:4 e :6; firebase.json, hosting.rewrites; src/config/env.js:54.

R15 — Fazer a suíte completa funcionar como barreira de publicação

Prioridade: P2 · Evidência: testes executados · Área: qualidade.

Problema e impacto: npm test termina com mensagem de testes pausados, sem validar o aplicativo. A execução real encontrou quatro falhas: dois testes gerais e dois de navegador. O CI atual executa testes de Node, mas não inclui lint, check:full ou a suíte de navegador. Os novos testes visuais não impedem sozinhos uma regressão na publicação.

Melhoria sugerida: atualizar testes que ainda esperam o cabeçalho antigo; corrigir o helper usado indevidamente no admin de tenant; alinhar o contrato PNG/JPG; tornar explícito e obrigatório o conjunto de verificações de cada alteração. Adicionar emulador para concorrência e persistência crítica.

Critério de aceite: o comando padrão realmente executa testes; CI valida backend, interface e segurança; falha bloqueia publicação; o relatório distingue erro do teste de defeito funcional.

Referências: package.json, scripts.test; .github/workflows/dev-pix-validation.yml; test/app.test.js:443; test/image-compression.test.js:197; test/browser/pdv.spec.js:27; test/browser/storefront.spec.js:136.

R16 — Compilar os recursos da interface antes da publicação

Prioridade: P2 · Evidência: leitura da página do caixa · Área: carregamento e previsibilidade.

Problema e impacto: o caixa carrega React e Babel Standalone por CDN, transforma JSX no navegador e usa Tailwind via CDN. As versões do React no HTML e no package.json pertencem a gerações diferentes. Assim, o arquivo de dependências instalado não descreve integralmente o que a interface executa.

Melhoria sugerida: compilar JSX e CSS no build, fixar dependências e gerar arquivos com hash; servir o pacote a partir do próprio domínio. Manter importações pesadas de mapa e relatórios sob demanda e medir o carregamento em celular.

Critério de aceite: o caixa abre sem compilador JSX no navegador; indisponibilidade desses CDNs não impede venda; build e testes usam o mesmo conjunto de versões da publicação.

Referências: src/http/views-caixa.html:31 e :81; package.json, scripts.build e dependencies.react.

R17 — Atualizar dependências com avaliação de compatibilidade

Prioridade: P2 · Evidência: auditoria do registro de pacotes · Área: dependências.

Problema e impacto: npm audit --omit=dev retornou 8 entradas moderadas na cadeia Firebase/Google Cloud/gaxios/uuid. As entradas não representam oito falhas independentes: o resultado aponta a cadeia relacionada ao aviso GHSA-w5hq-g745-h8pq. A revisão não demonstrou que a aplicação expõe o caminho vulnerável com buffers controlados pelo usuário.

Melhoria sugerida: avaliar a atualização dos pacotes diretos e suas dependências transitivas; testar Firestore, Storage, autenticação e tarefas em ambiente isolado. O resultado da auditoria indica que parte da solução sugerida envolve mudança de versão principal; não executar correção forçada sem análise.

Critério de aceite: o aviso é eliminado ou sua exceção tem justificativa, responsável e prazo; o lockfile é reproduzível; testes críticos passam com as versões atualizadas.

Referências: package-lock.json; .audit-review-dependencies.json; aviso https://github.com/advisories/GHSA-w5hq-g745-h8pq.

R18 — Preservar o último catálogo válido em falhas transitórias

Prioridade: P2 · Evidência: leitura do tratamento de erro · Área: catálogo e atendimento.

Problema e impacto: refreshCatalog limpa o catálogo com loadItems([]) em qualquer falha, antes de lançar o erro. O atendimento registra que usará o cache existente, mas esse cache acabou de ser apagado. Uma instabilidade do banco pode deixar busca e recomendações sem produtos mesmo após carregamento válido anterior.

Melhoria sugerida: substituir o cache somente depois de obter e validar uma nova leitura; conservar a última versão válida com data e estado de desatualização. Diferenciar falha transitória de bloqueio de autorização. Preço e estoque de uma venda continuam obrigatoriamente conferidos no servidor.

Critério de aceite: uma leitura válida seguida de timeout mantém o catálogo de consulta; uma leitura realmente vazia substitui o cache; comprar nunca usa saldo antigo como autorização de venda.

Referências: src/catalog/catalog.repository.js:29 e :36; src/services/message.service.js:76.

R19 — Unificar validação, erros e contrato de unidade

Prioridade: P2 · Evidência: leitura e reprodução de unidade · Área: contratos de dados.

Problema e impacto: o app aceita formatos diferentes para produto e unidade. unitOf dos rankings trata saleUnit como string, mas outros módulos usam saleUnit.code. Um produto com saleUnit: { code: 'KG' } pode aparecer como unidade. Além disso, o wrapper do PDV devolve 400 para qualquer exceção, inclusive falha de infraestrutura; isso confunde erro corrigível pelo operador com indisponibilidade.

Melhoria sugerida: normalizar DTOs na entrada e na leitura; validar tipos, limites e campos permitidos; ter um contrato único de dinheiro e unidade; mapear erros tipados para 400/401/403/404/409/429/503, com código estável e mensagem pública segura.

Critério de aceite: KG funciona nos formatos legados migrados; requisição inválida não gera 500; falta de banco gera indisponibilidade; conflito de e-mail é identificável sem analisar texto livre.

Referências: src/reports/product-sales-metrics.js:5; src/catalog/normalize-catalog-item.js:5; src/storefront/existing-store.js:17; src/http/pdv.routes.js:87.

R20 — Mostrar saúde real e tornar falhas operacionais rastreáveis

Prioridade: P2 · Evidência: leitura dos diagnósticos · Área: observabilidade.

Problema e impacto: o diagnóstico de WhatsApp chama de ONLINE uma configuração com token, telefone e verify token presentes, sem comprovar que a API aceita o token. O estado do webhook fica apenas em memória por instância. Logs também não possuem identificador de requisição consistente para ligar venda, tarefa, cobrança e mensagem.

Melhoria sugerida: separar configurado, conexão verificada e último processamento bem-sucedido; adotar IDs de correlação; redigir dados pessoais nos logs; alertar sobre fila atrasada, Pix tardio, erro de permissão e fechamento incompleto. Declarar e verificar agendamento do trabalhador e retenção de sessões/eventos.

Critério de aceite: token inválido não aparece como conexão verificada; um pedido pode ser rastreado até a notificação; alertas são testados; limpeza e recuperação do banco têm responsável e procedimento comprovado.

Referências: src/services/system-status.service.js:35; src/shared/logger.js:5; src/http/pdv.routes.js:454; src/security/web-session.js:46.

R21 — Reduzir os módulos que concentram muitas responsabilidades

Prioridade: P3 · Evidência: estrutura e tamanho dos arquivos · Área: manutenção.

Problema e impacto: app.js tem cerca de 2.952 linhas; o repositório principal, 3.792; o HTML administrativo, 2.273; e o componente principal do caixa, 1.891. Há ainda implementações paralelas de autenticação, checkout e catálogo entre a loja principal, os tenants e os fluxos legados. Mudanças de navegação já exigiram correções em vários testes e superfícies.

Melhoria sugerida: extrair por domínio, começando pelos pontos das correções anteriores; separar rotas, políticas, serviços e persistência; compartilhar componentes de botão, diálogo e erro; registrar quais fluxos legados ainda têm usuários. Introduzir tipos ou JSDoc verificado nos contratos antes de uma migração ampla.

Critério de aceite: cada extração mantém o comportamento comprovado por testes; regras financeiras não ficam em HTML; uma mudança comum tem uma origem principal; remoção de legado exige evidência de desuso.

Referências: src/http/app.js; src/database/firestore.repository.js; src/http/views-admin.js; public/pdv/app.jsx; src/storefront; src/components.

R22 — Evoluir o painel com indicadores que orientem decisões

Prioridade: P3 · Evidência: oportunidade de produto, sem defeito presumido · Área: gestão Conflora.

Situação: os quatro rankings e os líderes mensais já existem. A quantidade mistura medidas diferentes, e a sazonalidade observada depende do estoque disponível e do tempo de cadastro. Um item sem venda pode estar indisponível; faturamento alto também não significa margem alta.

Melhoria sugerida: permitir rankings separados de kg e unidades; distinguir produto sem venda de produto sem estoque; acrescentar custo, margem e perdas quando esses dados forem registrados. Exibir dias com estoque, cobertura estimada e comparação com o mesmo mês de anos anteriores. Manter poucas métricas principais e detalhes sob demanda.

Critério de aceite: cada indicador tem fórmula e fonte documentadas; falta de custo não aparece como lucro; rankings mostram período, unidade e disponibilidade; sugestões de reposição usam dados suficientes e não são apresentadas como previsão garantida.

Referências: src/http/views-admin-gerencial.js:540 e :578; src/reports/product-sales-metrics.js:8; src/reports/period-executive-metrics.js:128.

Ordem sugerida de execução

Etapa 1 — Proteção e integridade: tratar R01–R09 em alterações pequenas. Corrigir os pontos de HTML inseguro, credenciais, usuários padrão e PDFs públicos; proteger tarefas e ingressos; corrigir datas financeiras e idempotência. Só encerrar a etapa com testes negativos de acesso e repetição após falhas.

Etapa 2 — Persistência e publicação: executar R10–R15 e R19. Consolidar vínculos de clientes, confirmar apenas gravações duráveis, limitar listeners e leituras de sessão, unificar o destino da publicação e recuperar a suíte completa. Usar emulador do Firestore para testar concorrência; testes em memória não provam comportamento do banco real.

Etapa 3 — Desempenho e operação: executar R16–R18 e R20. Compilar a interface, atualizar dependências de forma controlada, corrigir o cache e criar diagnóstico operacional verificável. Medir carregamento no celular, leituras por requisição e idade das notificações pendentes antes e depois.

Etapa 4 — Evolução: executar R21–R22 progressivamente. Extrair módulos quando houver uma mudança real no domínio e validar os indicadores adicionais com o dono da empresa. Evitar uma reescrita geral, que ampliaria o risco sobre pagamentos e estoque já protegidos.

Sequência de dependências: HTML e contratos antes da política de conteúdo; hashes antes de retirar credenciais antigas; identidade única antes de consolidar saldos; fatos financeiros antes de agregar relatórios; build reproduzível antes de automatizar publicação. Preço, estoque, débito e baixa devem continuar conferidos pelo backend.

Rotina para cada entrega: registrar cenário atual; implementar uma correção; executar testes de regressão e falhas; validar no ambiente separado; revisar o resultado; publicar com plano de retorno; confirmar comportamento pelo domínio real. Este documento não autoriza nem executa publicação.

Evidências, testes e limites da revisão

Identificação: revisão do repositório em C:/Users/Junin/confloraai, commit 209edfec5f98da75abfb68fe03cee22b540e55ed. As reproduções adicionais iniciaram servidores temporários com NODE_ENV=test e dados em memória. O código do aplicativo não foi modificado.

Testes gerais desta sessão: 236 executados, 234 aprovados, 2 com falha. app.test.js:443 espera uma marcação antiga do cabeçalho. image-compression.test.js:197 espera arquivo JPG onde o fluxo preserva PNG; é preciso alinhar o contrato antes de decidir se o teste ou a implementação deve mudar. Registro: .audit-current-unit.log.

Navegador, revisão ampla: 70 executados, 68 aprovados, 2 com falha. pdv.spec.js:27 espera o nome acessível antigo do link inicial. storefront.spec.js:136 chama o helper do admin principal dentro do admin de tenant, onde esse menu não existe. Essas falhas não provam defeito nos pagamentos nem quebra do isolamento de lojas; deixam esses cenários sem validação completa. Registro: .audit-review-all-browser.log.

Verificação focada anterior desta sessão: 33 de 33 testes aprovados para navegação, largura do caixa, erro de e-mail, perfil, painel, catálogo e pagamentos. Sintaxe e lint também passaram. Registro: .audit-current-browser.log. Resultados verdes não cobrem automaticamente os problemas descobertos pelas reproduções de segurança.

Reproduções adicionais: senha preservada em texto = true; PDF sem login = HTTP 200; execução de evento do nome no admin = true; leitura negada retorna administrador de exemplo = true; venda criada em janeiro e paga em fevereiro = entrada de 100 em janeiro e 0 em fevereiro; nova tentativa após falha = DUPLICATE; tarefa sem segredo, com assistente ativo = HTTP 200; fila retorna inProcess = webhook HTTP 200 e zero mensagens processadas.

Dependências: npm audit --omit=dev consultado em 18/10/2026. Foram 8 entradas moderadas na cadeia de dependências de produção. Isso não avalia os scripts servidos diretamente por CDN nem comprova exploração. Registro: .audit-review-dependencies.json.

Limites: não foi realizado teste de invasão em produção, nem cobrança real, envio real de WhatsApp, inspeção de IAM, validação de regras publicadas ou restauração de backup. Regras restritivas de desenvolvimento existem; não se deve inferir delas o estado publicado de produção. Infraestrutura, desempenho sob carga e recuperação precisam de validação específica antes de serem considerados aprovados.